

MYJAVA MGR & CONNECTION POOL REST API DOCUMENTATION

계층형 지출 관리 시스템

myJava Mgr REST API 종합 명세서

myJava(DBConnectionMgr + XxxMgr + XxxBean) 백엔드 RESTful 웹 서비스 명세서

36

Java 클래스

10

XxxMgr 클래스

13

XxxBean 클래스

44

API 엔드포인트

위치: C:\Java\expense-mgr-api | 패턴: DBConnectionMgr (Pool) + Mgr + Bean | Base URL:
http://localhost:8080

목 차

0 프로젝트 개요 & ERD

myJava Mgr 아키텍처 · DBConnectionMgr 분석 ·
전체 ERD

1 회원 관리 (User)

UserBean · UserMgr · UserMgrHandler

2 지출 관리 (Expense)

ExpenseBean · ExpenseMgr ·
ExpenseMgrHandler

3 카테고리 (Category)

LargeCategoryBean · MediumCategoryBean ·
CategoryMgr

4 예산 관리 (Budget)

BudgetBean · BudgetMgr · BudgetMgrHandler

5 포인트 시스템 (Point)

PointHistoryBean · PointMgr · PointMgrHandler

6 커뮤니티 (Community)

PostBean · PostMgr · PostMgrHandler

7 친구 관리 (Friend)

FriendBean · FriendMgr · FriendMgrHandler

8 팀 챌린지 (Challenge)

TeamRoomBean · TeamMemberBean ·
ChallengeMgr

9 상점 & 앱 설정 모듈

ItemBean · PurchaseBean · StoreMgr ·
SettingMgr

10 myJava Mgr REST API 구동 및 44개 API 전수 JSON 예시

MgrApiServer · 44개 엔드포인트 명세표 · 44개 API
전수 JSON 예시 카드

CHAPTER 0

프로젝트 개요 & myJava Mgr 아키텍처

이 프로젝트는 `C:\Java\myJava` 폴더의 전통적인 **DBConnectionMgr** (커넥션 풀 싱글톤) + **XxxMgr** + **XxxBean** 패턴 기반의 초경량 REST API 백엔드입니다.

아키텍처 패턴

DBConnectionMgr

- 싱글톤 커넥션 풀
- `con = pool.getConnection()`
- `pool.freeConnection(con, pstmt, rs)`

XxxMgr (10개)

- UserMgr, ExpenseMgr 등
- `Vector<XxxBean>` 반환 방식
- PreparedStatement SQL 처리

XxxBean (13개)

- UserBean, ExpenseBean 등
- JavaBean 규격 (Getter/Setter)
- JSON 자동 직렬화

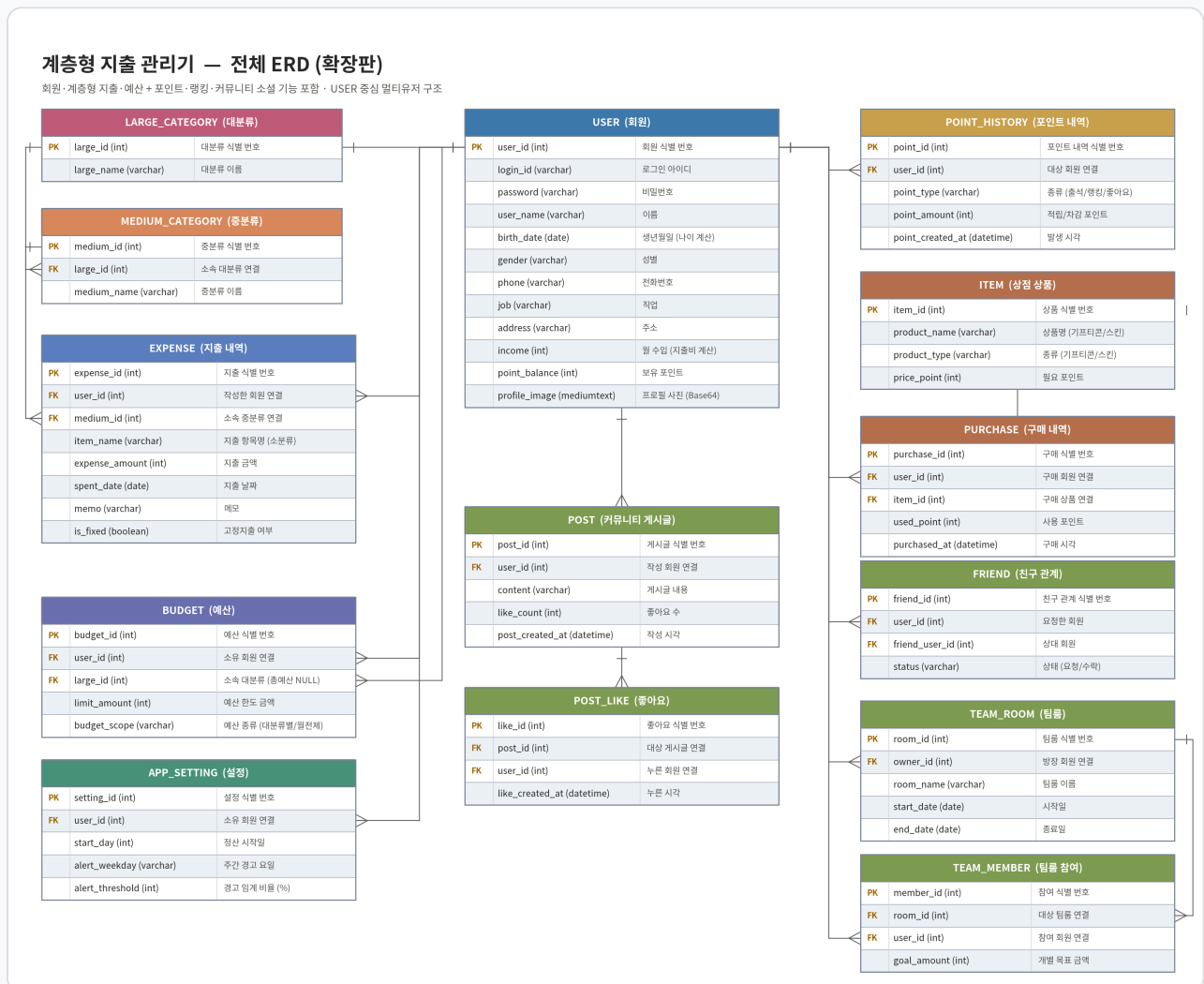
전체 DB 테이블 구조

테이블	역할	주요 컬럼	연관 클래스
<code>user</code>	회원 정보	<code>user_id</code> , <code>login_id</code> , <code>password</code> , <code>income</code> , <code>point_balance</code>	UserMgr / UserBean / UserMgrHandler
<code>expense</code>	지출 내역	<code>expense_id</code> , <code>user_id</code> , <code>medium_id</code> , <code>item_name</code> , <code>expense_amount</code> , <code>is_fixed</code>	ExpenseMgr / ExpenseBean / ExpenseMgrHandler
<code>large_category</code>	지출 대분류	<code>large_id</code> , <code>large_name</code>	CategoryMgr / LargeCategoryBean / CategoryMgrHandler
<code>medium_category</code>	지출 중분류	<code>medium_id</code> , <code>medium_name</code> , <code>large_id</code>	CategoryMgr / MediumCategoryBean / CategoryMgrHandler
<code>budget</code>	예산 설정	<code>budget_id</code> , <code>user_id</code> , <code>large_id</code> (NULL=전체), <code>limit_amount</code>	BudgetMgr / BudgetBean / BudgetMgrHandler
<code>app_setting</code>	앱 알림 설정	<code>setting_id</code> , <code>user_id</code> , <code>start_day</code> , <code>alert_weekday</code> , <code>alert_threshold</code>	SettingMgr / AppSettingBean / SettingMgrHandler
<code>point_history</code>	포인트 적립/차감	<code>point_id</code> , <code>user_id</code> , <code>point_type</code> , <code>point_amount</code>	PointMgr / PointHistoryBean / PointMgrHandler
<code>item</code>	상점 상품 마스	<code>item_id</code> , <code>product_name</code> , <code>product_type</code> , <code>price_point</code>	StoreMgr / ItemBean / StoreMgrHandler

터

purchase	구매 내역	purchase_id, user_id, item_id, used_point	StoreMgr / PurchaseBean / StoreMgrHandler
post	커뮤니티 게시물	post_id, user_id, content, like_count	PostMgr / PostBean / PostMgrHandler
post_like	게시글 좋아요	like_id, post_id, user_id	PostMgr / PostLikeBean / PostMgrHandler
friend	친구 관계	friend_id, user_id, friend_user_id, status(요청/수락)	FriendMgr / FriendBean / FriendMgrHandler
team_room	챌린지 팀룸	room_id, owner_id, room_name, start_date, end_date	ChallengeMgr / TeamRoomBean / ChallengeMgrHandler
team_member	팀룸 참여자	member_id, room_id, user_id, goal_amount	ChallengeMgr / TeamMemberBean / ChallengeMgrHandler

전체 ERD (Entity Relationship Diagram)



CHAPTER 1

회원 관리 (User)

로그인, 회원가입, 마이페이지 정보 수정, 비밀번호 변경, 포인트 잔액 갱신을 담당합니다.

UserBean

UserMgr

UserMgrHandler

BEAN

UserBean

USER 테이블 데이터 전달 객체 (JavaBean)

필드명	타입	컬럼	설명
userId	int	user_id (PK)	회원 식별 번호 (AUTO_INCREMENT)
loginId	String	login_id	로그인 아이디 (중복 불가)
password	String	password	비밀번호
userName	String	user_name	이름
income	int	income	월 수입
pointBalance	int	point_balance	보유 포인트 잔액
profileImage	String	profile_image	프로필 사진 (Base64 인코딩, 선택)

CHAPTER 2

지출 관리 (Expense)

지출 내역 등록/수정/삭제/조회 및 총 지출 합계를 담당합니다.

ExpenseBean

ExpenseMgr

ExpenseMgrHandler

BEAN

ExpenseBean

지출 내역 자바 빈즈

필드명	타입	설명
<code>expenseId</code>	int	지출 식별 번호 (PK)
<code>userId</code>	int	회원 ID (FK)
<code>mediumId</code>	int	중분류 ID (FK)
<code>itemName</code>	String	지출 항목명
<code>expenseAmount</code>	int	지출 금액 (원)
<code>spentDate</code>	Date	지출 날짜
<code>fixed</code>	boolean	고정지출 여부
<code>largeName</code> / <code>mediumName</code>	String	JOIN 표시 전용 대/중분류명

CHAPTER 10

myJava Mgr REST API 종합 명세표 & JSON 요청/응답 예시

Base URL: `http://localhost:8080` (서버 실행법 + 종합 명세표 + 44개 API 전수 예시 카드)

REST API 서버 실행 명령어

POWERSHELL에서 실행

1. Maven Exec 플러그인으로 실행

```
C:\Java\apache-maven-3.9.6\bin\mvn.cmd exec:java -f C:\Java\expense-mgr-api\pom.xml
```

2. Executable JAR로 실행

```
java -jar C:\Java\expense-mgr-api\target\expense-mgr-api-1.0.0.jar
```

3. IDE에서 실행

```
src/main/java/api/MgrApiServer.java 우클릭 → Run (실행)
```

전체 REST API 엔드포인트 종합 명세표 (10개 모듈 · 44개 API)

Method	URL Path	기능 명칭 및 설명	요청 파라미터 / Body 데이터
--------	----------	------------	--------------------

1. 회원 관리 (UserMgrHandler) — /api/users

2. 지출 관리 (ExpenseMgrHandler) — /api/expenses

3. 카테고리 관리 (CategoryMgrHandler) — /api/categories

4. 예산 관리 (BudgetMgrHandler) — /api/budgets

7/17

Method	URL Path	기능 명칭 및 설명	요청 파라미터 / Body 데이터
5. 포인트 시스템 (PointMgrHandler) — /api/points			
GET	/api/points/balance	포인트 잔액 조회	Query Param: ?userId=1
GET	/api/points/history	포인트 적립/사용 내역	Query Param: ?userId=1
POST	/api/points/earn	포인트 적립 (+amount)	JSON Body: {"userId":1, "pointType":"출석", "amount":10}
6. 커뮤니티 (PostMgrHandler) — /api/posts			
GET	/api/posts	전체 게시글 피드 목록	-
POST	/api/posts	신규 게시글 작성	JSON Body: PostBean 객체
POST	/api/posts/{postId}/like	게시글 좋아요	Query Param: ?userId=1
7. 친구 관리 (FriendMgrHandler) — /api/friends			
GET	/api/friends	수락 완료된 친구 목록	Query Param: ?userId=1
POST	/api/friends/request	친구 요청 보내기	JSON Body: FriendBean 객체
8. 팀 챌린지 (ChallengeMgrHandler) — /api/challenges			
GET	/api/challenges/rooms	전체 팀룸 목록 조회	-
POST	/api/challenges/rooms	팀룸(챌린지) 신규 생성	JSON Body: TeamRoomBean
POST	/api/challenges/rooms/join	팀룸 참가 신청	JSON Body: TeamMemberBean
9. 상점 (StoreMgrHandler) — /api/store			
GET	/api/store/items	전체 상품 목록	-
POST	/api/store/buy	포인트 상품 구매	JSON Body: {"userId":1, "itemId":2}
10. 앱 설정 (SettingMgrHandler) — /api/settings			
GET	/api/settings	회원 앱 설정 정보 조회	Query Param: ?userId=1
POST	/api/settings	앱 설정 등록/수정	JSON Body: AppSettingBean 객체
11. 프로필 관리 (ProfileMgrHandler) — /api/profile			
POST	/api/profile	프로필 사진 업데이트	JSON Body: {"userId":1, "profileImage":"base64 ... "}

 전체 JSON 요청/응답 예시**POST** 1-1. POST /api/users/login (로그인)

요청 BODY (JSON)

```
{"loginId": "woojin", "password": "password123"}
```

응답 (HTTP 200 OK - USERBEAN)

```
{"userId": 1, "loginId": "woojin", "userName": "정우진", "phone": "010-1234-5678",  
"income": 3500000, "pointBalance": 1250}
```

POST 1-2. POST /api/users/register (신규 회원가입)

요청 BODY (JSON)

```
{"loginId": "jimin", "password": "pw1234!@", "userName": "한지민", "phone": "010-9999-8888",  
"income": 2000000, "profileImage": "iVBORw0KGgoAAAANSU..."} 
```

응답 (HTTP 201 CREATED)

```
{"success": true, "message": "회원가입이 성공적으로 완료되었습니다."}
```

GET 1-3. GET /api/users/check-duplicate?loginId=woojin (아이디 중복확인)

응답 (HTTP 200 OK)

```
{"loginId": "woojin", "duplicate": true, "message": "이미 사용 중인 아이디입니다."}
```

GET 1-4. GET /api/users/1 (회원 상세정보 조회)

응답 (HTTP 200 OK - USERBEAN)

```
{"userId": 1, "loginId": "woojin", "userName": "정우진", "income": 3500000, "pointBalance": 1250}
```

PUT 1-5. PUT /api/users/1 (회원 정보 수정)

요청 BODY (JSON)

```
{"userName": "정우진", "phone": "010-5555-7777", "income": 4000000}
```

응답 (HTTP 200 OK)

```
{"success": true, "message": "회원정보가 수정되었습니다."}
```

PUT 1-6. PUT /api/users/1/password (비밀번호 변경)

요청 BODY (JSON)

```
{"newPassword": "newPassword5678!"}
```

응답 (HTTP 200 OK)

```
{"success": true, "message": "비밀번호가 변경되었습니다."}
```

GET 2-1. GET /api/expenses?userId=1 (지출 목록)

응답 (HTTP 200 OK - EXPENSEBEAN 배열)

```
[{"expenseId": 101, "userId": 1, "largeName": "식비", "mediumName": "카페/디저트", "itemName": "스타벅스 아메리카노", "expenseAmount": 4500}]
```

GET**2-2. GET /api/expenses/101 (지출 단건 상세)**

응답 (HTTP 200 OK)

```
{"expenseId": 101, "userId": 1, "mediumId": 3, "itemName": "스타벅스 아메리카노",  
"expenseAmount": 4500}
```

POST**2-3. POST /api/expenses (지출 신규 등록)**

요청 BODY (JSON)

```
{"userId": 1, "mediumId": 3, "itemName": "투썸 케이크", "expenseAmount": 6500,  
"spentDate": "2026-08-12"}
```

응답 (HTTP 201 CREATED)

```
{"success": true, "message": "지출이 등록되었습니다."}
```

PUT**2-4. PUT /api/expenses/101 (지출 수정)**

요청 BODY (JSON)

```
{"mediumId": 3, "itemName": "스타벅스 라떼", "expenseAmount": 5000}
```

응답 (HTTP 200 OK)

```
{"success": true, "message": "지출이 수정되었습니다."}
```

DELETE**2-5. DELETE /api/expenses/101 (지출 삭제)**

응답 (HTTP 200 OK)

```
{"success": true, "message": "지출이 삭제되었습니다."}
```

GET 2-6. GET /api/expenses/total?userId=1 (총 지출 합계)

응답 (HTTP 200 OK)

```
{"userId": 1, "totalExpense": 635000}
```

GET 2-7. GET /api/expenses/stats/category?userId=1&startDay=25 (대분류 집계 %)

응답 (HTTP 200 OK)

```
[{"largeName": "식비", "totalAmount": 285000, "percentage": 44.9}, {"largeName": "교통", "totalAmount": 150000, "percentage": 23.6}]
```

GET 3-1. GET /api/categories/large (전체 대분류)

응답 (HTTP 200 OK)

```
[{"largeId": 1, "largeName": "식비"}, {"largeId": 2, "largeName": "교통/차량"}]
```

GET 3-2. GET /api/categories/medium/by-large/1 (하위 중분류)

응답 (HTTP 200 OK)

```
[{"mediumId": 1, "largeId": 1, "mediumName": "장보기"}, {"mediumId": 3, "largeId": 1, "mediumName": "카페/디저트"}]
```

GET 4-1. GET /api/budgets?userId=1 (예산 목록)

응답 (HTTP 200 OK)

```
[{"budgetId": 11, "userId": 1, "largeId": 1, "limitAmount": 500000, "budgetScope": "식비"}]
```

POST 4-2. POST /api/budgets (카테고리별 예산 등록)

요청 BODY (JSON)

```
{"userId": 1, "largeId": 2, "limitAmount": 300000, "budgetScope": "교통"}
```

응답 (HTTP 201 CREATED)

```
{"success": true, "message": "예산이 등록되었습니다."}
```

POST 4-3. POST /api/budgets/total (총 예산 등록/수정)

요청 BODY (JSON)

```
{"userId": 1, "limitAmount": 1000000, "budgetScope": "전체"}
```

응답 (HTTP 200 OK)

```
{"success": true, "message": "총 예산이 성공적으로 반영되었습니다."}
```

GET 5-1. GET /api/points/balance?userId=1 (포인트 잔액)

응답 (HTTP 200 OK)

```
{"userId": 1, "pointBalance": 1250}
```

GET 5-2. GET /api/points/history?userId=1 (포인트 내역)

응답 (HTTP 200 OK)

```
[{"pointId": 1, "userId": 1, "pointType": "출석체크", "amount": 10, "createdAt": "2026-08-25"}]
```

POST 5-3. POST /api/points/earn (포인트 적립)

요청 BODY (JSON)

```
{"userId": 1, "pointType": "출석체크", "amount": 10}
```

응답 (HTTP 200 OK)

```
{"success": true, "message": "10 포인트가 적립되었습니다."}
```

GET 6-1. GET /api/posts (전체 피드)

응답 (HTTP 200 OK)

```
[{"postId": 5, "userId": 2, "content": "이번 주 외식 안하기 성공!", "likeCount": 12}]
```

POST 6-2. POST /api/posts (게시글 작성)

요청 BODY (JSON)

```
{"userId": 1, "content": "오늘부터 가계부 작성 시작합니다!"}
```

응답 (HTTP 201 CREATED)

```
{"success": true, "message": "게시글이 등록되었습니다."}
```

GET 7-1. GET /api/friends?userId=1 (수락 친구 목록)

응답 (HTTP 200 OK)

```
[{"friendId": 1, "userId": 1, "friendUserId": 2, "status": "수락",  
  "friendUserName": "김지현"}]
```

POST 7-2. POST /api/friends/request (친구 요청)

요청 BODY (JSON)

```
{"userId": 1, "friendUserId": 2}
```

응답 (HTTP 201 CREATED)

```
{"success": true, "message": "친구 요청을 보냈습니다."}
```

GET 8-1. GET /api/challenges/rooms (전체 팀룸 목록)

응답 (HTTP 200 OK)

```
[{"roomId": 1, "ownerId": 1, "ownerName": "정우진", "roomName": "8월 절약왕",  
  "startDate": "2026-08-01", "endDate": "2026-08-31"}]
```

POST 8-2. POST /api/challenges/rooms (팀룸 생성)

요청 BODY (JSON)

```
{"ownerId": 1, "roomName": "30만원 식비 줄이기", "startDate": "2026-08-01",  
  "endDate": "2026-08-31"}
```

응답 (HTTP 201 CREATED)

```
{"success": true, "message": "팀룸이 생성되었습니다."}
```

POST 8-3. POST /api/challenges/rooms/join (팀룸 참가)

요청 BODY (JSON)

```
{"userId": 1, "goalAmount": 4000000}
```

응답 (HTTP 200 OK)

```
{"success": true, "message": "팀룸에 참가하셨습니다."}
```

GET 9-1. GET /api/store/items (상품 목록)

응답 (HTTP 200 OK)

```
[{"itemId": 1, "productName": "스타벅스 쿠폰", "pricePoint": 4500}]
```

POST 9-2. POST /api/store/buy (상품 구매)

요청 BODY (JSON)

```
{"userId": 1, "itemId": 1}
```

응답 (HTTP 200 OK)

```
{"success": true, "message": "상품 구매 성공!"}
```

GET 10-1. GET /api/settings?userId=1 (앱 설정 조회)

응답 (HTTP 200 OK)

```
{"settingId": 1, "userId": 1, "startDay": "2026-08-25", "alertWeekday": "MON",  
"alertThreshold": 80}
```

POST 10-2. POST /api/settings (앱 설정 등록/수정)

요청 BODY (JSON)

```
{"userId": 1, "startDay": "2026-08-25", "alertWeekday": "FRI", "alertThreshold":  
90}
```

응답 (HTTP 200 OK)

```
{"success": true, "message": "앱 설정이 성공적으로 변경되었습니다."}
```

POST 11-1. POST /api/profile (프로필 사진 업데이트)

요청 BODY (JSON)

```
{"userId": 1, "profileImage": "data:image/png;base64,iVBORw0KGgoAAAANSU..."}
```

응답 (HTTP 200 OK)

```
{"success": true, "message": "프로필 사진이 성공적으로 변경되었습니다."}
```